

AI POWERED PREDICTIVE MODEL FOR PERSONALIZED GUIDANCE OF STUDENTS

A Thesis submitted in partial fulfillment of the requirements

for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

by

Pratham Bansal (2100681530037)

Divyansh Vats (2100681530019)

Varun Goel (2100681530057)

Under the Supervision of

Mrs. Mohini Preetam Singh, Assistant Professor

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



**MEERUT INSTITUTE OF ENGINEERING AND TECHNOLOGY,
MEERUT 250005**



DR. A.P.J. ABDUL KALAM TECHNICAL UNIVERSITY, U.P., LUCKNOW

APRIL, 2025

DECLARATION

We hereby declare that this submission is our work and that, to the best of our knowledge and belief, it contains no material previously published or written by another person nor material which to a substantial extent has been accepted for the award of any other degree or diploma of the university or other institute of higher learning, except where due acknowledgment has been made in the text.

Signature:

Name : Pratham Bansal

Roll No. : 2100681530037

Date :

Signature:

Name : Divyansh Vats

Roll No. : 2100681530019

Date :

Signature:

Name : Varun Goel

Roll No. : 2100681530057

Date :

CERTIFICATE

This is to certify that *Project Report entitled —Predictive Model For Personalized Guidance Of Students* which is submitted by *Pratham Bansal (2100681530037)*, *Divyansh Vats (2100681530019)*, *Varun Goel (2100681530057)* in partial fulfillment of the requirement for the award of degree B. Tech. in Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning) of Dr. A.P.J. Abdul Kalam Technical University, U.P., Lucknow., is a record of the candidates own work carried out by them under my supervision. The matter embodied in this Project report is original and has not been submitted for the award of any other degree.

Date:

Supervisor

ACKNOWLEDGEMENTS

It gives us a great sense of pleasure to present the report of the B. Tech Project undertaken during B.Tech. Final Year. We owe a special debt of gratitude to our guide (*Assistant Professor*) *Mohini Singh*, Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), Meerut Institute of Engineering and Technology, Meerut for her constant support and guidance throughout the course of our work. Her sincerity, thoroughness and perseverance have been a constant source of inspiration for us. It is only her cognizant efforts that our endeavors have seen light of the day.

We also do not like to miss the opportunity to acknowledge the contribution of all faculty members of the department for their kind assistance and cooperation during the development of our project. Last but not the least, we acknowledge our friends for their contribution in the completion of the project.

Signature :

Name

Roll No. :

Date :

Signature:

Name:

Roll No:

Date:

Signature :

Name :

Roll No. :

Date :

ABSTRACT

Modern academic education often follows a "one size-fits-all" approach, which fails to accommodate the diverse capabilities and limitations of individual students. This study addresses this issue by building a system that generates personalized learning pathways based on students' past academic performance. Leveraging ensemble learning techniques, the system classifies students into distinct categories to offer personalized guidance within the boundaries of the standard curriculum and syllabus. While fully personalized learning styles may not be immediately feasible in the current educational framework, this approach seeks to balance individual needs with practical implementation. Using a publicly available student performance data set from Kaggle, extensive data cleaning and preprocessing were performed before applying various machine learning algorithms, like K-Nearest Neighbors, Linear Regression, Decision Trees and Naive Bayes. Additionally, ensemble learning model such as Random Forest was implemented to enhance classification accuracy. Students were categorized into ten distinct classes based on the predictive analysis of these models. Personalized guidance reports in PDF format were then generated for students based on their predicted class, providing actionable recommendations tailored to their needs. Furthermore, the system incorporates an SQLite database to store user details and inputs each time they interact with the system, ensuring data persistence and future usability. The study highlights the superiority of ensemble learning methods over individual machine learning models in terms of classification accuracy and reliability. This work aims to serve as a foundation for integrating AI-driven personalized guidance systems in mainstream education, ensuring effective learning experience for students.

TABLE OF CONTENT

	Page No.
DECLARATION.....	i
CERTIFICATE.....	ii
ACKNOWLEDGEMENTS.....	iii
ABSTRACT.....	iv
LIST OF TABLES	v
LIST OF FIGURES	vi
CHAPTER 1 -INTRODUCTION	1
1.1 INTRODUCTION.....	1
1.2 PROBLEM STATEMENT.....	2
1.3 SCOPE.....	2
CHAPTER 2- LITRATURE REVIEW.....	4
CHAPTER 3- METHODOLOGY USED.....	5
3.1 DATA COLLECTION	
3.2 DATA CLEANING	
3.3 PREPROCESSING	
3.4 BINNING	
3.5 MODEL EVALUATION AND SELECTION	
3.6 ALGORITHM TO CREATE PERSONALIZED REPORTS	
3.4.1 USER INTERFACE (IF ANY).....	
	19
CHAPTER 4- TECHNOLOGY USED.....	
4.1 FRAMEWORK	
4.2 LIBRARIES	
	19
CHAPTER 5- RESULTS & DISCUSSIONS	
CHAPTER 6- CONCLUSION & FUTURE WORK	
	23
REFERENCES.....	24
APPENDIX.....	26

LIST OF TABLES

S. No.	Description	Page No.
3.3.1	Summary of the data preprocessing process	6
3.5.1	Table 2: Accuracy Table	8

LIST OF FIGURES

S.NO	DESCRIPTION	PAGE NO.
1	Workflow Of the Project	11
3.3.1	Box Plot Before Scaling	6
3.4.1	Box Plot After Scaling	7
3.5.1	Accuracy of Different Models	9
3.5.2	KNN	12
3.5.3	RANDOM FOREST	14
3.5.4	SVM	15
3.5.5	Decision Tree	15
3.5.6	Naive Bayes Classifier	16
5.1	Student Performance prediction Form	21
5.2	Personalized Guidance PDF report	22

1. CHAPTER 1: INTRODUCTION

1.1 INTRODUCTION

Academic education is undergoing a rapid evolution with the integration of advanced technologies, particularly in data analytics and machine learning. These technologies have made it feasible to analyze large datasets, uncover insights, and address challenges that were previously difficult to tackle. However, modern academic systems predominantly adhere to a "one-size-fits-all" approach, eliminating the unique learning capacities and limitations of individual students. Despite significant advancements in educational technology, achieving meaningful personalization within mainstream education remains a complex challenge.

Personalization is vital to ensure that students receive tailored guidance to assess, understand, and improve their learning processes, while also identifying and addressing areas where they lag. Previous research has explored personalized learning through machine learning, focusing on designing customized courses, study styles, and materials based on students' strengths and interests. While innovative, these methods often lack scalability and practical applicability, particularly in countries like India, where education is bound by rigid curriculum and syllabus mandated by school boards and universities. Students in these systems have limited freedom to modify their course structure or replace compulsory subjects with those aligned with their future goals. As a result, even motivated learners are constrained from pursuing subjects that resonate with their interests.

This study aims to bridge this gap by offering personalized guidance within the boundaries of predefined curriculum, enabling students to improve their performance in weaker subjects while maintaining excellence in areas of strength. The proposed approach involves identifying the most correlated features influencing student performance through data mining techniques. Classification algorithms such as K-Nearest Neighbors, Support Vector Machines, Decision Trees, and Naive Bayes and Random Forest are employed to categorize students into ten distinct performance classes. Based on these classifications, personalized guidance reports in PDF format are generated for each student, providing actionable insights for improvement. The novelty of this work lies in its focus on delivering personalized guidance while adhering to the constraints of existing curriculum. Furthermore, the use of ensemble learning models, such as Random Forest, enhances the accuracy of classification compared to individual machine learning models. Prior research has consistently demonstrated the superiority of ensemble methods for numerical classification tasks.

Unlike existing studies that primarily emphasize predicting student performance using machine learning algorithms, this study takes a step further by integrating these predictions into actionable outcomes. The methodology encompasses data collection, cleaning, preprocessing, binning, classification model evaluation, and the generation of personalized PDF reports. The backend implementation utilizes Flask, coupled with an SQLite database to store user data and maintain records for future comparisons and longitudinal analysis. By addressing both practical constraints and individual learning needs, this study contributes to the development of scalable, curriculum-aligned personalized guidance systems. It offers a blueprint for integrating artificial intelligence into education to foster inclusive and adaptive learning environments.

1.2 PROBLEM STATEMENT

Education systems today largely follow a fixed curriculum, leaving little room for personalized learning. Every student has unique strengths and weaknesses, but the traditional "one-size-fits-all" approach doesn't account for these differences. While technology and machine learning have introduced new ways to customize education, most solutions lack scalability, especially in rigid academic structures like those in India. Students often struggle to improve in weaker subjects while excelling in their strengths due to limited flexibility in their learning paths. This project aims to tackle this challenge by using machine learning to analyze student performance and generate personalized guidance, helping them improve within the constraints of the existing curriculum.

1.3 SCOPE

The integration of machine learning and data analytics is transforming education, making it possible to analyze student performance and provide personalized learning experiences. However, mainstream education still follows a rigid, standardized approach that fails to accommodate individual learning needs. This project aims to bridge this gap by developing a **personalized guidance system** that works within the constraints of existing curricula.

Focus Areas

1. Need For Personalization

- Personalized learning helps students identify and improve weaker areas while excelling in their strengths.
- Previous research on machine learning in education has focused on customizing study materials but lacks practical scalability.
- In rigid education systems like India's, students have minimal flexibility in modifying their curriculum.

2. Objective Of the Project

- Develop a system that provides personalized academic guidance based on student performance.
- Ensure that recommendations align with the structured curriculum.
- Use machine learning techniques to classify students into different performance categories.

Approach and Methodology

3. Data Analysis and Feature Identification

- Identify key factors influencing student performance through data mining.
- Analyze student data to extract meaningful insights.

4. Machine Learning Techniques used

- Classification algorithms employed include:
 - A. K-Nearest Neighbors
 - B. Support Vector Machines
 - C. Decision Trees
 - D. Naive Bayes
 - E. Random Forest (Ensemble Learning)
- Random Forest improves accuracy compared to individual models.

5. Personalized Guidance System

- Students are categorized into **ten performance classes** based on ML classification results.
- Based on which class they are classified the model generates a custom PDF reports are generated with insights and recommendations.

6. Implementation Details

- **Backend:** Flask framework
- **Database:** SQLite (for storing student data and tracking progress)
- **Process:**
 1. Data collection and preprocessing
 2. Performance classification using ML models
 3. Generation of personalized learning reports

CHAPTER 2 : LITERATURE REVIEW

Personalized learning has garnered significant attention in educational research, particularly with the advent of machine learning techniques. Researchers such as [1] and [3] have developed adaptive learning systems that provide customized courses, study materials, and instructional styles based on students' strengths and interests. While these methods have shown promise, they often face challenges in scalability, especially in countries like India, where rigid curriculum and standardized syllabus limit the flexibility required for personalized course design [3]. As a result, the practical implementation of such systems in mainstream education has been limited. Various machine learning models have been applied to predict and analyze student performance. Studies have employed Decision Trees, Naive Bayes, and K-Nearest Neighbors (KNN)[5],[6] to classify students based on academic records and behavioral attributes. While these models are effective in identifying trends and predicting outcomes, they often lack mechanisms for providing actionable feedback to students. Ensemble learning models, such as Random Forest and Gradient Boosting, have been highlighted in studies[11],[14],[15] for their superior accuracy and robustness in handling classification tasks involving numerical data. However, their application in education has primarily focused on performance prediction rather than delivering personalized guidance.

Some research focused on selection of optimal classification algorithms based on the nature of datasets and the number and type of features that datasets hold [2]. Others focused on selection and identification of most relevant features for predicting the performance and data mining techniques[4],[12]. These work, although helpful, does not give personalized guidance. Despite these advancements, a significant gap remains in aligning personalized learning solutions with rigid curricular boundaries. Most existing studies have focused on performance prediction or providing personalized resources without addressing the constraints of standardized syllabus. This study addresses this gap by offering personalized guidance to students while adhering to their prescribed curriculum.

Several studies have also investigated the use of deep learning models in educational research. Deep learning techniques, particularly neural networks, have demonstrated promising results in predicting student learning behaviors and outcomes. For example, recurrent neural networks (RNNs) and long short-term memory (LSTM) models have been successfully applied to track students' learning progress over time, identifying trends and potential areas of improvement [13]. However, these models often require extensive computational resources and large datasets, which can limit their scalability in real-world applications.

Recent advancements in Natural Language Processing (NLP) have also contributed to personalized learning research. NLP-based recommendation systems have been explored for generating automated feedback on student assignments, analyzing textual responses, and offering tailored study suggestions [8]. These techniques help in bridging the gap between generic feedback and highly specific, student-centric guidance. However, such approaches still need refinement to align with rigid academic structures and standardized grading systems.

Another emerging trend in personalized learning is the integration of AI-driven chatbots and virtual tutors. Studies have shown that AI-powered chatbots can provide real-time academic assistance, answer queries, and offer study recommendations based on a student's past performance and learning patterns [9]. While these systems improve student engagement and learning efficiency, they often lack the depth and adaptability required to cater to diverse learning needs across different educational boards.

Additionally, some studies have explored the potential of reinforcement learning (RL) in adaptive education. RL-based systems have been designed to optimize learning paths by continuously adjusting recommendations based on student interactions [10]. Unlike static recommendation systems, RL-driven models offer dynamic, personalized suggestions that evolve over time. However, implementing such models in structured academic settings remains a challenge due to curriculum constraints and limited adaptability of traditional education frameworks.

Gamification in education has also been studied as an alternative approach to enhancing student engagement and learning outcomes. Researchers have explored the use of gamified learning environments where students earn rewards and progress through adaptive levels based on their academic performance [7]. While gamification has been found to increase motivation and participation, its direct impact on long-term knowledge retention and standardized academic performance is still an area of ongoing research.

Furthermore, collaborative filtering and social learning networks have been investigated to facilitate peer-assisted learning. By analyzing peer interactions and performance patterns, these models recommend study groups, shared resources, and discussion forums to improve student understanding [6]. However, such systems require strong data privacy mechanisms to ensure the confidentiality of student information.

Despite the progress in AI-driven education, a persistent challenge is ensuring ethical AI usage, particularly in data privacy and fairness. Studies highlight the risks of algorithmic bias, where machine learning models may unintentionally favor certain demographic groups, leading to unfair learning outcomes [15]. Addressing these ethical concerns is crucial to developing truly equitable and effective personalized learning systems.

In conclusion, while significant strides have been made in leveraging machine learning and AI for personalized education, existing solutions still face limitations in scalability, curriculum constraints, and ethical considerations. This study builds on prior research by offering a structured, curriculum-aligned personalized guidance system that balances individual learning needs with practical implementation. By integrating classification models, predictive analytics, and personalized feedback mechanisms, this work aims to bridge the gap between traditional education and AI-driven personalization.

CHAPTER 3: METHODOLOGY USED

This study employs a structured methodology to design and implement a personalized learning system by integrating data preprocessing, feature selection, classification models, and ensemble learning techniques, aiming to provide actionable guidance for students within the boundaries of their curriculum.

3.1 DATA COLLECTION

We took the Student performance prediction dataset from kaggle with dimensions of 10000 rows and 6 columns. The 10000 rows represent the data of 10000 students and 6 columns represent 6 parameters or attributes about every student. This is a labelled dataset which means it is a dataset suitable for supervised learning algorithm. The six features are “Hours Studied”, “Previous Scores”, “Extracurricular Activities”, “Sleep Hours”, “Sample Question paper attempted” and “Performance Index”.

3.2 DATA CLEANING

We will load the dataset in our colab working environment and check if there are any NULL , NAN , missing or redundant values or a column in our dataset. We have to remove these data inconsistencies as these negatively impact our model performance at the time of model training and testing. First we will check null or missing values , if the null or missing values in the dataset are less than we can remove them either by removing and ignoring that particular row. Columns that contain more than 50% null values can be dropped as they would not be reliable for the model. But, if the null values are less than 50%, we can replace the missing values with the mean, median or most frequent value by a simple imputer.

3.3 PREPROCESSING

Data preprocessing is a critical step in ensuring the quality and reliability of the dataset used for analysis. In this study, preprocessing involved handling removing duplicates, and normalizing numerical data to maintain uniformity across features. Additionally, categorical data were encoded to make them compatible with machine learning algorithms, and outliers were addressed to improve model performance. Feature scaling techniques, such as standardization and normalization, were applied to ensure that all features contributed equally to the classification process. These steps ensured that the dataset was clean, consistent, and suitable for training and testing the models.

Table 3.3.1: Summary of the data preprocessing process

STEPS	TOOLS/TECHNIQUES REQUIRED
HANDLING MISSING VALUES	Pandas, SimpleImputer (sklearn)
REMOVING DUPLICATES	Pandas
ENCODING CATEGORICAL DATA	LabelEncoder (sklearn)
OUTLIER DETECTION	IQR method, Z-score
FEATURE SCALING	StandardScaler, MinMaxScaler (sklearn)
DATA SPLITTING	train_test_split (sklearn)
DATA TRANSFORMATION	Log transformation

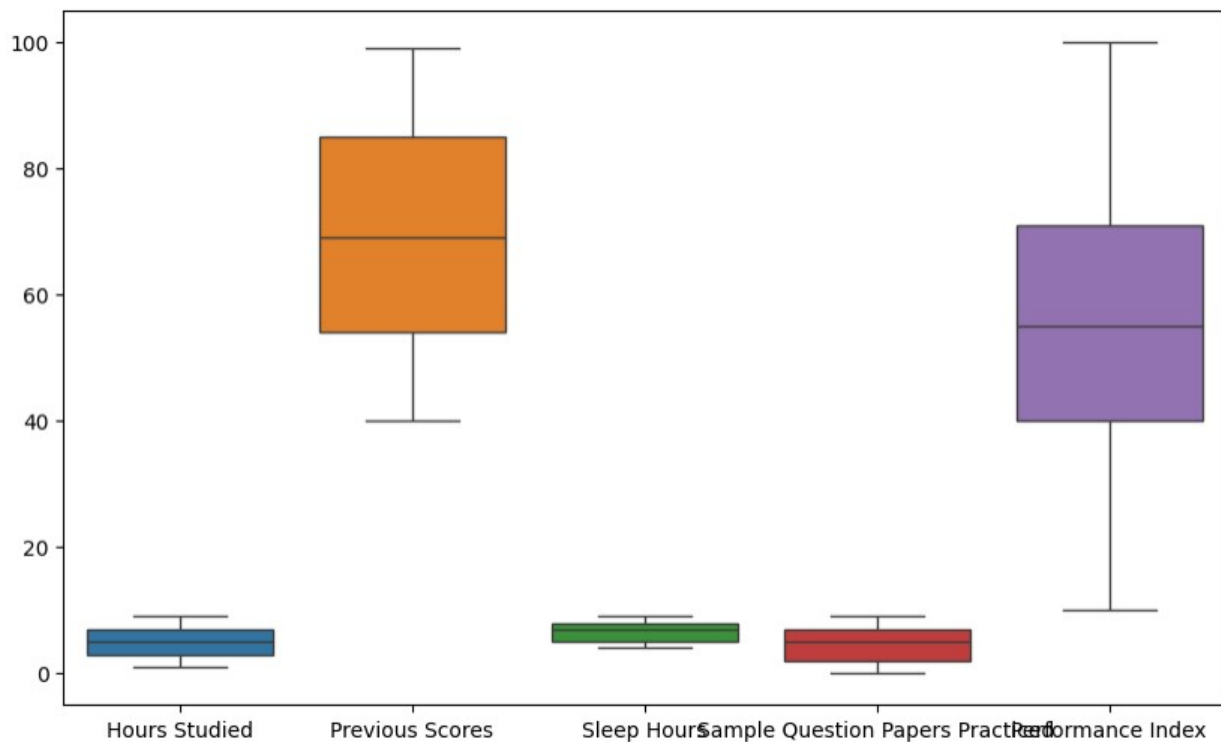


Figure 3.3.1 : Box Plot before Scaling

3.4 BINNING

Data preprocessing is divided into various techniques that simplify the analyst's work and increase the accuracy of the outcomes in machine learning. Binning is one of the approaches required to reduce the influence of continuous variables and, at the same time, allows forming large volumes of data according to specific categorizations. In case the researcher has hundreds of rows to be analyzed, he/she may create dozens of bins instead of scanning and overthinking every variable separately. Binning helps to convert the data in the continuous form to the structured one.

In this study, the **Performance Index** column, which ranges from 0 to 100, was divided into 10 equal bins, each representing a range of 10 values (e.g., 0–10, 11–20, and so on). This binning approach allowed us to classify students into distinct categories based on their performance. Each bin corresponds to a specific performance group, enabling targeted personalized guidance for students within each range.

By dividing the data into bins, we simplified the classification process and enhanced the interpretability of results, aligning with the study's objective of providing actionable insights for improving student performance.

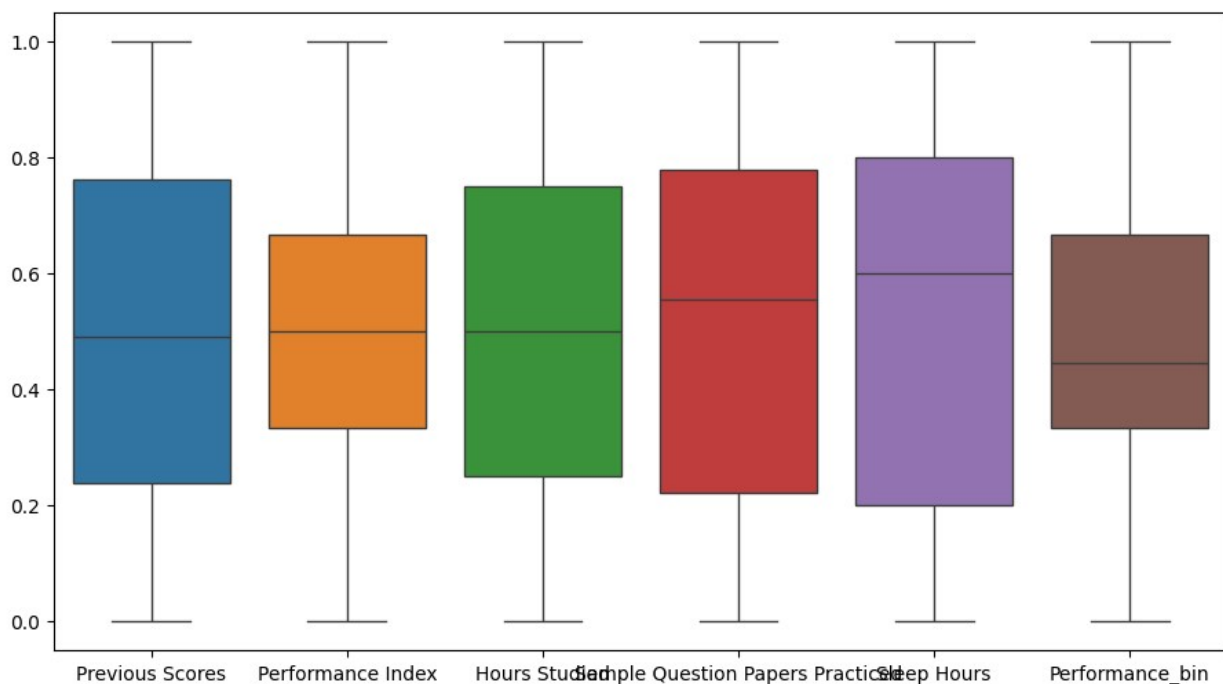


Figure 3.4.1 : Box Plot after Scaling

3.5 MODEL EVALUATION AND SELECTION

Model evaluation and selection play a critical role in identifying the most effective algorithm for classifying student performance and generating personalized guidance. To ensure a comprehensive comparison, we first evaluated several individual machine learning models, followed by ensemble learning techniques.

Finally, individual models were evaluated on the processed dataset, with their performance being gauged in terms of classification accuracy. The KNN classifier performed well on the dataset and achieved an accuracy of **95.54%**. We then realized an intermediary model: a Naive Bayes classifier as follows, which boosted the accuracy to **95.74%**. The Support Vector Machine (SVM) yielded a much lower accuracy score of **93.9%** largely because the performance of the model is sensitive to the distribution of the data. While both individual models achieved high accuracy the Decision Tree classifier was the most suitable with an accuracy of **95.9%**. Yet, we have also seen that use of ensemble learning technique, such as with Random Forest, gives us the highest accuracy we have achieved, **97.21%**.

This result highlights the strength of ensemble models in aggregating the predictions of multiple decision trees, reducing overfitting, and improving overall classification performance. The results demonstrate that while individual models perform well, ensemble learning provides a significant boost in accuracy, making **Random Forest** the optimal choice for this study.

Table3.5.1: Accuracy Table

MODEL	ACCURACY (%)
K-Nearest Neighbors	95.54
Naive Bayes	95.74
Support Vector Machine	93.9
Decision Tree	95.9
Random Forest (Ensemble)	97.21

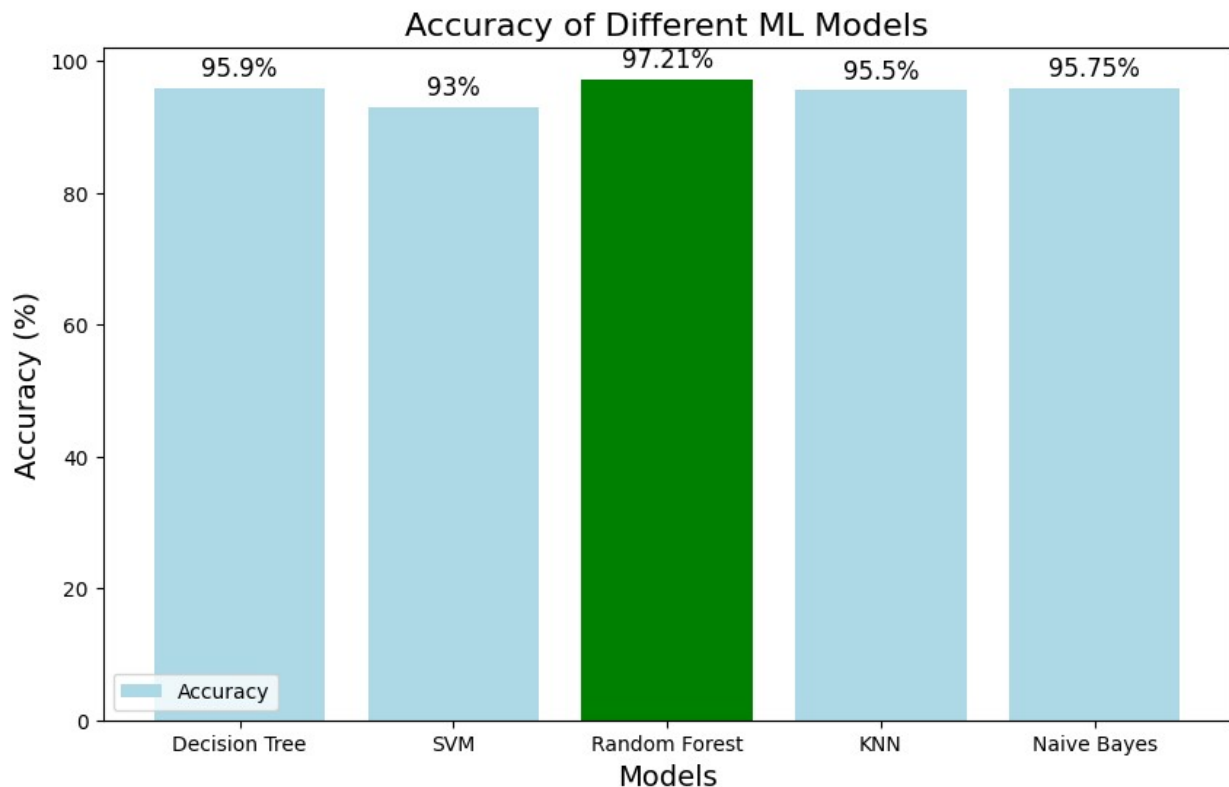


Figure 3.5.1: Accuracy of Different Models

K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a simple yet powerful machine learning algorithm that falls under the category of **non-parametric** and **lazy learning** models. Being **non-parametric** means it does not make any prior assumptions about the distribution of data, making it highly adaptable for real-world datasets that do not necessarily follow strict mathematical patterns. On the other hand, KNN is referred to as **lazy learning** because it does not build a model during the training phase. Instead, it stores all training data and uses it only during the prediction stage, which speeds up training but makes the testing phase slower and more resource-intensive. This is because, during testing, the algorithm must scan the entire dataset to find the closest neighbors, requiring significant memory and processing power in worst-case scenarios.

KNN Working

The "**K**" in KNN represents the number of nearest neighbors considered when classifying a data point. The choice of **K** plays a critical role in determining the algorithm's accuracy. Typically, an **odd value of K** is preferred when dealing with binary classification problems to avoid ties.

For instance, if we have a point **P1** whose label needs to be determined, the algorithm identifies the **K** closest data points using distance metrics such as:

Euclidean Distance (for continuous variables)

Manhattan Distance (useful in grid-like data)

Hamming Distance (for categorical variables)

Minkowski Distance (a generalized distance metric)

Once the nearest neighbors are found, the class label is assigned based on the majority vote among them. Essentially, each of the **K** neighbors casts a vote for its respective class, and the class receiving the highest number of votes is assigned to **P1**.

Eager vs. Lazy Learners

Machine learning algorithms can generally be classified into **eager** and **lazy learners**:

Eager learners build a generalized model from the training data before making predictions. These models are proactive, meaning they process the data beforehand so that predictions can be made quickly.

Lazy learners like KNN, on the other hand, do not perform any model training. They simply store the dataset and delay computation until a prediction is required. These algorithms compare new data points directly with stored instances to determine their classification. While this approach allows flexibility, it also makes the classification process slower.

The Curse of Dimensionality

KNN works efficiently when dealing with a **small number of features**, but as the number of dimensions increases, performance tends to degrade. This is known as the **Curse of Dimensionality**—a phenomenon where higher dimensions lead to data sparsity, making it harder to compute meaningful distances between points.

To mitigate this issue, **dimensionality reduction techniques** such as **Principal Component Analysis (PCA)** or **feature selection methods** can be applied before using KNN. Additionally, research suggests that in high-dimensional spaces, distance-based measures like Euclidean distance become less effective. Alternative similarity measures, such as **cosine similarity**, may perform better in such cases.

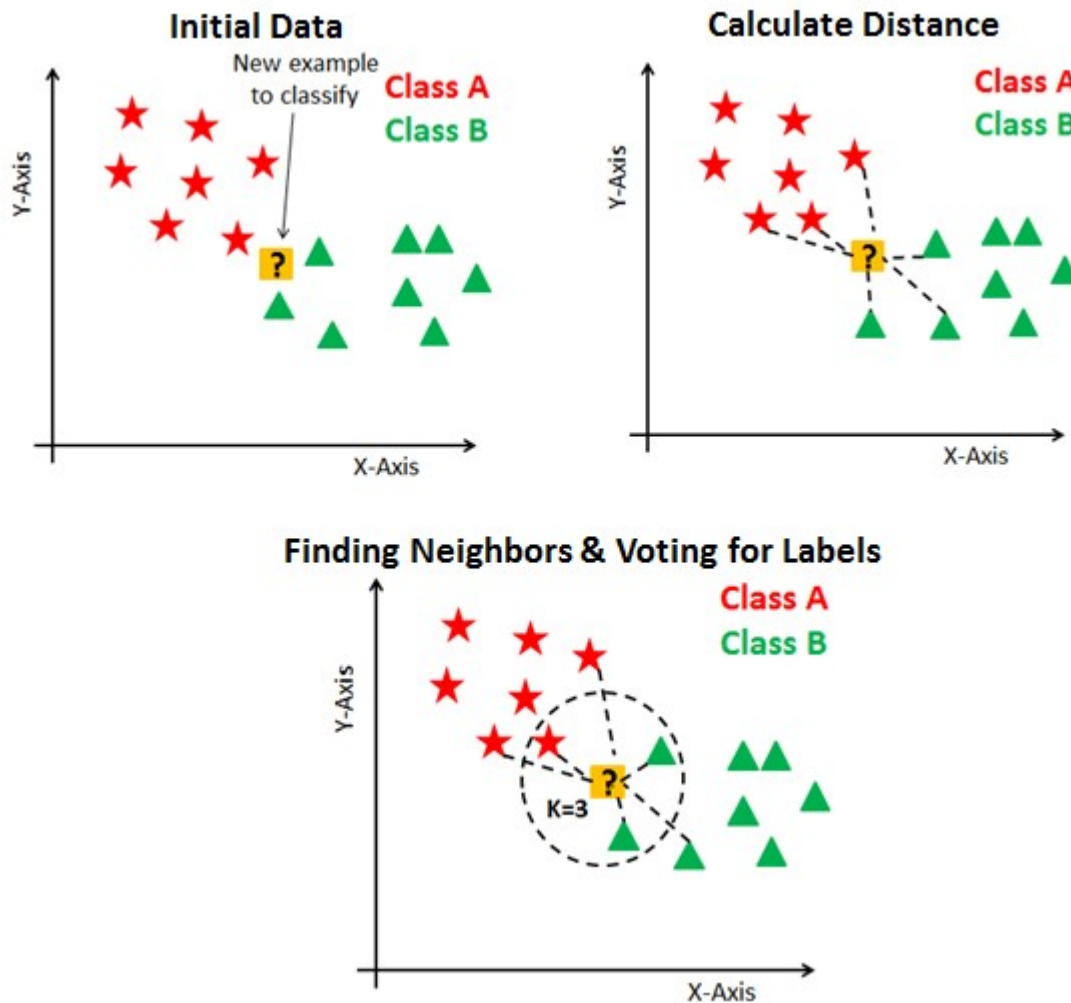


Figure:3.5.2 KNN Algorithm

RANDOM FOREST

Random Forest is a widely used and highly effective machine learning algorithm that excels in both **classification** and **regression** tasks. As an **ensemble learning method**, it constructs multiple decision trees and merges their outputs to enhance accuracy while minimizing the risk of overfitting. This article explores how Random Forest works, its advantages and limitations, and its real-world applications.

What is a Random Forest?

A Random Forest is essentially a **collection of decision trees**, each trained on different subsets of the dataset. Unlike a single decision tree that can be prone to overfitting, Random Forest improves generalization by combining multiple trees. The final output is determined through:

Majority voting (for classification)

Averaging predictions (for regression)

By aggregating the results of multiple trees, the algorithm produces more **reliable** and **stable** predictions compared to using a single decision tree.

Random Forest Working

The Random Forest algorithm follows these key steps:

1. Bootstrap Sampling (Bagging)

Multiple **bootstrap samples** are created by randomly selecting data points **with replacement** from the original dataset.

Some data points may appear more than once in a sample, while others may be excluded.

Each decision tree is trained on a different bootstrap sample.

2. Random Feature Selection

Instead of considering all features at each decision node (as in standard decision trees), Random Forest selects a **random subset** of features.

This introduces additional randomness, reducing correlation among trees and improving the model's ability to generalize.

3. Growing Decision Trees

Each tree is allowed to grow **fully** without pruning.

Since the trees are trained on different subsets of data, they may overfit to their respective samples, but when combined, the ensemble remains robust.

4. Aggregation of Predictions

For classification, each tree makes a prediction, and the final class is determined by majority voting.

For regression, the final output is obtained by averaging the predictions of all trees.

Random Forest

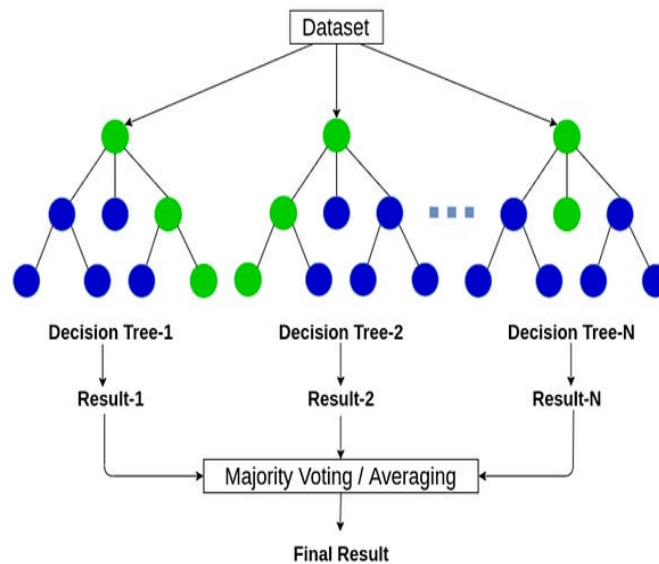


Figure3.5.3 Random Forest

SVM

Support Vector Machine (SVM) is a widely used **supervised learning algorithm** that excels in **classification** tasks, though it can also handle regression problems.

Although it can also handle regression issues, the popular supervised learning algorithm Support Vector Machine (SVM) performs best on classification tasks.

Finding the ideal decision boundary that divides data points into distinct classes within an n-dimensional space is the main goal of support vector machines (SVM). Future accurate classification of new data points is guaranteed by this boundary, also referred to as the hyperplane.

Support vectors, the crucial information points that determine the hyperplane's position, are what distinguish SVM from other algorithms. SVM is a strong and effective classification algorithm because of these support vectors, which help to maximize the margin between classes. The algorithm's name, Support Vector Machine, comes from the existence of these crucial points.

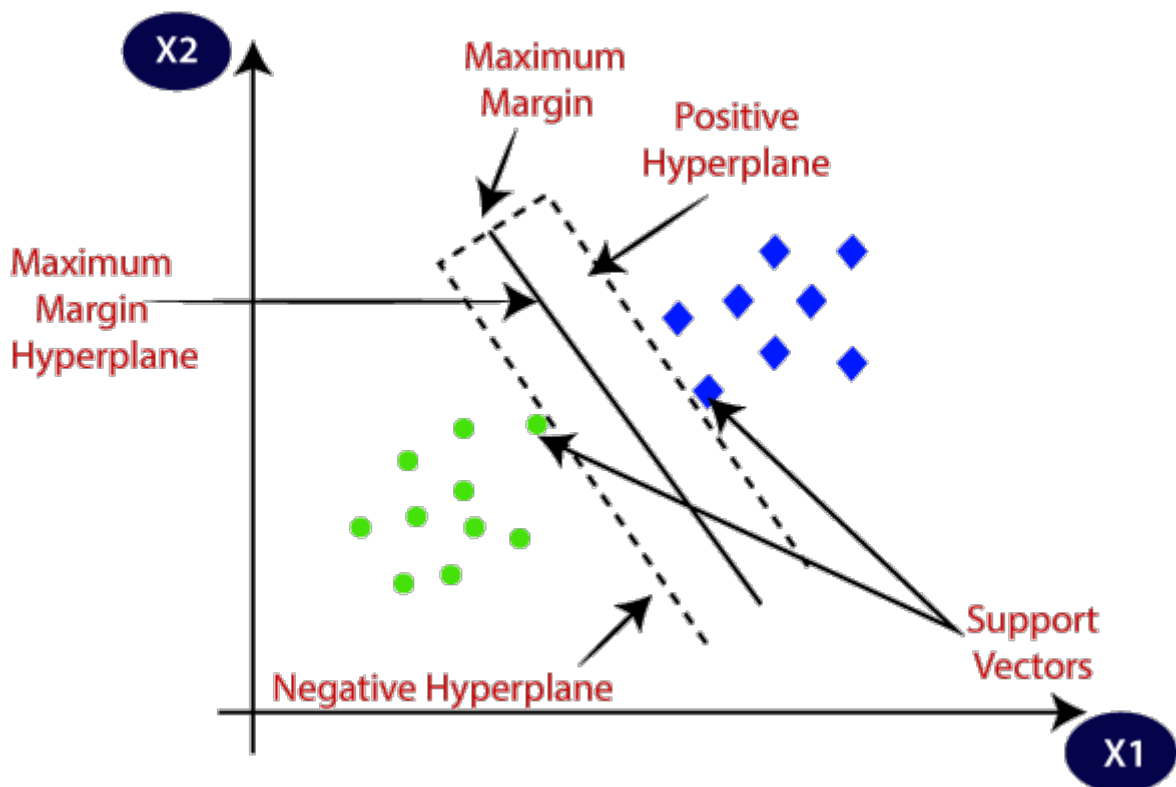


Figure 3.5.4 SVM

Decision Tree

Decision trees are **flexible and powerful models** used for both **classification and regression** tasks. Decision trees are versatile and effective models that can be applied to both regression and classification problems. Decision trees use a non-parametric approach, which allows them to capture intricate, nonlinear patterns in the data, in contrast to traditional linear models that presume a direct relationship between input variables and the target.

Decision trees' adaptability to real-world issues is one of their main advantages. They are extensively utilized in a variety of sectors where managing intricate decision-making procedures is essential, such as healthcare, finance, e-commerce, and manufacturing.

A decision tree's main goal is to methodically divide the dataset into smaller subsets until each group has little variation or is made up of similar data points. Decision trees are very useful for modeling because of this methodical approach.

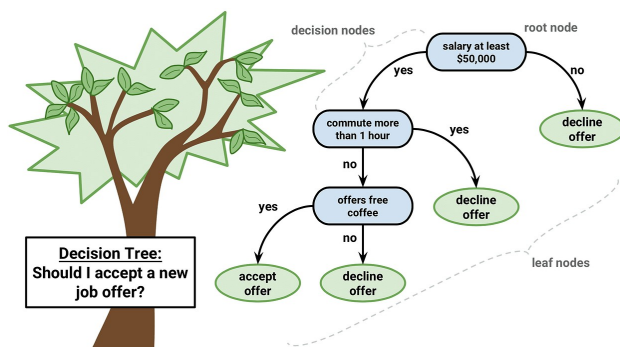


Figure 3.5.5 Decision Tree

Naive Bayes Algorithm

Two key ideas are the inspiration for the Naive Bayes algorithm's name. "Naive" describes the presumption that every feature in the dataset is unrelated to every other feature, i.e., that the existence of one feature has no bearing on the existence of another. The "Bayes" part refers to Bayes' Theorem, which forms the basis of this algorithm and aids in the computation of probabilities based on past information.

In order to use Naive Bayes for prediction, the dataset must first be transformed into a frequency table that summarizes the frequency of each feature. The likelihood that particular features will occur is then calculated to create a probability table. Finally, the posterior probability—which aids in the classification of new data points—is calculated using Bayes' Theorem.

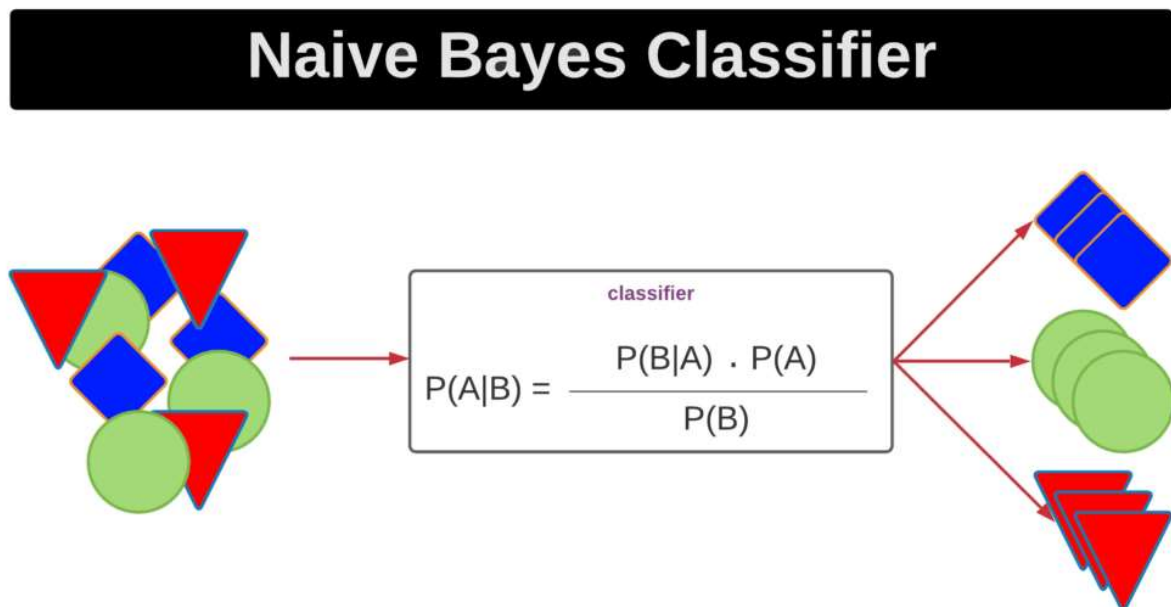


Figure 3.5.6 NB Classifier

3.6 ALGORITHM TO CREATE PERSONALIZED REPORTS

Creating personalized reports for students involves an algorithmic approach that integrates the classification results from the machine learning model with domain-specific insights. These reports are designed to provide actionable feedback and tailored recommendations to help students improve their academic performance while adhering to their curriculum constraints.

The steps in the algorithm are as follows:

1. Input Data:

Accept the student's input data, such as their performance metrics (e.g., scores, attendance, extracurricular activities). Validate the input data to ensure all required fields are filled and within acceptable ranges.

2. Classification:

Pass the preprocessed data through the trained Random Forest model to classify the student into one of the 10 performance bins. Retrieve the class label corresponding to the student's performance.

3. Recommendation Retrieval:

Use a predefined database of recommendations tailored to each performance class. Based on the student's class label, fetch relevant recommendations, such as study tips, areas requiring focus, and suggested resources.

4. Performance Insights:

Provide an analysis of the student's current strengths and weaknesses. Highlight subjects or topics where improvement is necessary.

5. Generate Personalized Report:

Design a structured PDF report for the student that includes: Current performance overview. Suggested study strategies. Recommended time allocation for weak subjects. Encouragement and motivational tips for better score.

6. Storage and Future Reference:

Save the report in the SQLite database along with the student's input data and predicted class for future comparisons and longitudinal tracking.

7. Delivery:

Provide a downloadable PDF to the student or deliver it via email.

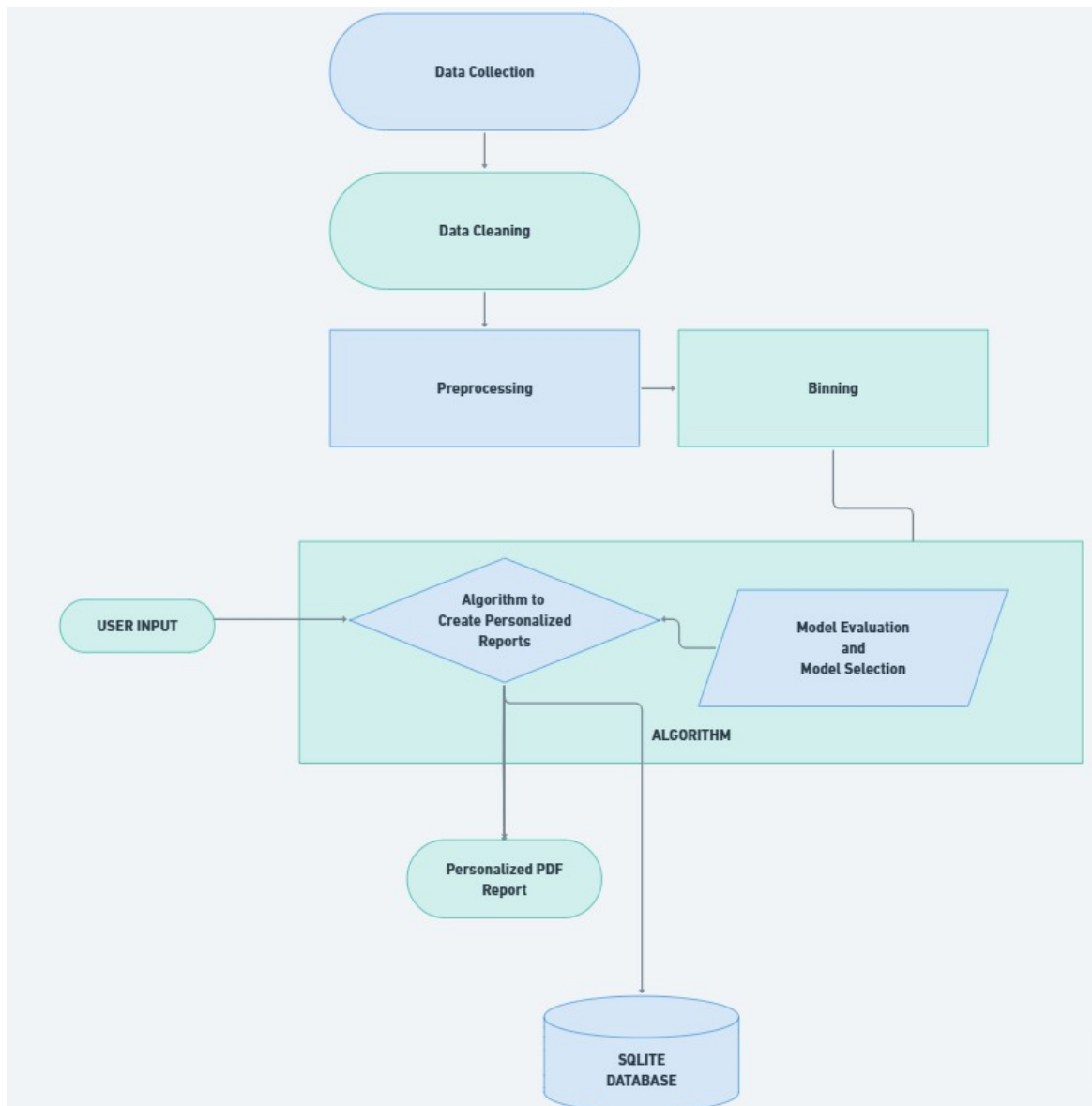


Figure 1: Flowchart or Workflow

CHAPTER 4- TECHNOLOGY USED

4.1 FRAMEWORK

- FLASK
- SCI-KIT LEARN
- XG-BOOST

4.2 LIBRARIES

- NUMPY
- PANDAS
- MATPLOTLIB
- SEABORN

CHAPTER 5- RESULTS & DISCUSSIONS

The results of this study showcase the effectiveness of using machine learning algorithms and ensemble learning techniques to classify students into performance bins and provide personalized guidance reports. The process was evaluated through various stages, from model evaluation and selection to the generation of actionable insights, and the findings are outlined below.

A. Model Performance

The first stage involved evaluating individual machine learning models on the dataset. The **K-Nearest Neighbors (KNN)** model achieved an accuracy of **95.54%**, showcasing its ability to classify students based on proximity metrics effectively. **Naive Bayes**, known for its probabilistic approach, slightly improved the accuracy to **95.74%**, while **Support Vector Machines (SVM)**, despite being effective for binary classification problems, performed relatively lower with an accuracy of **93.9%**. The **Decision Tree** classifier showed robust performance with an accuracy of **95.9%**, indicating its strength in handling categorical data. However, the **Random Forest**, an ensemble learning model, outperformed all individual algorithms with a superior accuracy of **97.21%**, validating its selection for this study.

B. Feature Relevance and Binning

The performance index, ranging from 0 to 100, was divided into 10 bins, each representing a range of 10 values (e.g., 0–10, 11–20, and so on). This binning approach enabled the classification of students into distinct groups for targeted guidance. Feature selection during preprocessing ensured that only the most relevant variables contributed to the classification, enhancing model accuracy and interpretability.

C. Personalized Guidance Reports

For each student classified into a performance bin, a personalized guidance report was generated. The reports included:

- (i) A summary of the student's current performance.
- (ii) Identification of weak areas requiring focus.
- (iii) Recommendations for study strategies, resources, a.
- (iv) Motivational tips and encouragement to help improve performance.

These reports were dynamically tailored based on the predicted performance class and provided students with actionable feedback while aligning with the curriculum constraints.

D. Storage and Accessibility

Each student's data and corresponding personalized report were stored in an SQLite database for future reference. This setup ensures that progress can be tracked over time and allows educators and students to compare current performance with historical data.

E. Insights

The study highlights the potential of using machine learning to bridge the gap between general curriculum-based education and the need for personalized learning pathways. The results indicate that ensemble learning models, particularly Random Forest, are more effective than individual machine learning algorithms for this classification task.

Student Performance Prediction

Student Name

Hours Studied

Previous Scores (scaled)

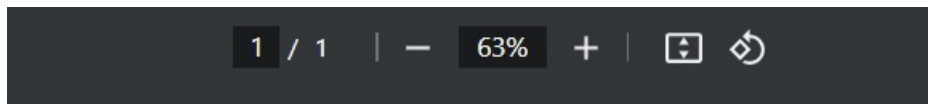
Extracurricular Activities (hours/week)

Sleep Hours (per day)

Sample Question Papers Practiced

Predict Performance

Figure 5.1: Student Performance prediction Form



Student Performance Report for Divya

Predicted Performance Level: 2

Personalized Guidance:

Good work! Keep practicing additional sample papers to strengthen your preparation.

Additional tips for further improvement:

- Review mistakes from previous tests to avoid repeating them.
- Increase focus on time management during tests.
- Explore advanced resources to challenge yourself.
- Maintain a healthy balance between study and relaxation.

Motivational Insights:

- Success is a journey, not a destination. Stay consistent in your efforts.
- Remember to celebrate small achievements along the way.
- Challenges are opportunities to grow and improve. Embrace them with confidence.
- Stay curious and keep learning beyond academics.

Generated on: 2024-12-26 10:50:12

Figure 5.2 : Personalized Guidance PDF report

CHAPTER 6- CONCLUSION & FUTURE WORK

This study was aimed at using machine learning and ensemble learning techniques to give personalized guidance to the students on the basis of their performance. This brings us to the goal of creating a system that not only classes students into appropriate performance bins, but also generates some actionable insights and recommendations about how they improve with the curriculum and syllabus defined. These findings suggest the potential utility of machine learning algorithms as effective tools for personalized education. The performance of multiple models such as K-Nearest Neighbors (KNN), Naive Bayes, Support Vector Machine (SVM), and Decision Tree were analyzed. While individual models like KNN and Naive Bayes offered reasonable accuracy, it was the Random Forest ensemble learning model that outperformed all others with an accuracy of 97.21%. This highlights the power of ensemble methods in reducing model bias and improving classification accuracy, especially when dealing with complex datasets.

The study also implemented a data binning technique, which divided students' performance into 10 distinct categories based on the range of scores. This enabled the creation of personalized reports for each student, providing insights into their strengths and weaknesses. The personalized reports, which included study tips, resource recommendations, and motivational guidance, were designed to improve student outcomes while adhering to the limitations of a fixed curriculum. This approach adds value by helping students focus on areas requiring improvement, without violating the constraints of formal education systems. In addition to the educational benefits, the system's integration with an SQLite database allowed for the storage and retrieval of student data and performance reports, ensuring that progress could be tracked over time. The ability to store and access historical data ensures that future interventions and support can be personalized and data driven. The findings of this study contribute to the growing body of research on personalized learning. While previous studies have focused on creating personalized curriculums or course recommendations, our work differentiates itself by focusing on providing personalized guidance within the boundaries of existing curricula. This makes the system more feasible for practical implementation, especially in countries like India, where students must follow a standardized syllabus.

In conclusion, this study proves that machine learning models, particularly ensemble learning techniques like Random Forest, can be successfully applied to personalize education and improve student performance.

The future potential of this system lies in its scalability, enabling its application to larger datasets and diverse educational contexts, making personalized learning accessible to more students globally. The system can be further refined by incorporating additional features and exploring deeper integration with other educational tools and platforms.

REFERENCES

- [1]W. Villegas-Ch, J. García-Ortiz and S. Sánchez-Viteri, "Personalization of Learning: Machine Learning Models for Adapting Educational Content to Individual Learning Styles," in *IEEE Access*, vol. 12, pp. 121114-121130, 2024, doi: 10.1109/ACCESS.2024.3452592.
- [2]Yadav, Nitin & Deshmukh, Sonal. (2023). Prediction of Student Performance Using Machine Learning Techniques: A Review. Doi: 10.2991/978-94-6463-136-4_63.
- [3]Siyu Wu, Yang Cao, JiaJun Cui, RunZe Li, Hong Qian, Bo Jiang, and Wei Zhang. 2024. A Comprehensive Exploration of Personalized Learning in Smart Education: From Student Modeling to Personalized Recommendations. *J. ACM* 37, 4, Article 111 (August 2024), 82 pages. <https://doi.org/10.48550/arXiv.2402.01666>
- [4]Iddrisu, Issah & Appiah, Obed & Appiahene, Peter & Inusah, Fuseini. (2023). A systematic review of the literature on machine learning application of determining the attributes influencing academic performance. *Decision Analytics Journal*. 7. 100204. 10.1016/j.dajour.2023.100204.
- [5]Mandalapu, Varun & Chen, Lujie & Shetty, Sushruta & Chen, Zhiyuan & Gong, Jiaqi. (2022). Student-centric Model of Learning Management System Activity and Academic Performance: from Correlation to Causation. 10.48550/arXiv.2210.15430.
- [6]Altabrawee, Hussein & Ali, Osama & Qaisar, Samir. (2019). Predicting Students' Performance Using Machine Learning Techniques. *JOURNAL OF UNIVERSITY OF BABYLON for pure and applied sciences*. 27. 194-205. 10.29196/jubpas.v27i1.2108.
- [7]Hussain, S., Khan, M.Q. Student-Performulator: Predicting Students' Academic Performance at Secondary and Intermediate Level Using Machine Learning. *Ann. Data. Sci.* **10**, 637–655 (2023). <https://doi.org/10.1007/s40745-021-00341-0>
- [8]Albreiki, Balqis, Nazar Zaki, and Hany Alashwal. 2021. "A Systematic Literature Review of Student' Performance Prediction Using Machine Learning Techniques" *Education Sciences* 11, no. 9: 552. <https://doi.org/10.3390/educsci11090552>
- [9]Ojajuni, Opeyemi & Ayeni, Foluso & Akodu, Olagunju & Ekanoye, Femi & Adewole, Samson & Ayo, Timothy & Misra, Sanjay & Mbarika, Victor. (2021). Predicting Student Academic Performance Using Machine Learning. 10.1007/978-3-030-87013- 3_36.
- [10]Communications on Applied Electronics (CAE) – ISSN : 2394- 4714 Foundation of Computer Science FCS, New York, USA Volume 7 – No. 11, December 2017 – www.caeaccess.org
- [11]Saleem, F.; Ullah, Z.;Fakieh, B.; Kateb, F. Intelligent Decision Support System for Predicting Student's E-Learning Performance Using Ensemble Machine Learning. *Mathematics* **2021**,9, 2078. <https://doi.org/10.3390/math9172078>
- [12]Ketui, Nongnuch & Wisomka, Warawut & Homjun, Kanitha. (2019). Using Classification Data Mining Techniques for Students Performance Prediction. 359-363. 10.1109/ECTI NCON.2019.8692227.

[13] Hindawi Computational Intelligence and Neuroscience Volume 2022, Article ID 5624475, 7 pages, <https://doi.org/10.1155/2022/5624475>

[14] a, I.N., Emiro, L., Bravo, C., Alejandro, J., Silva, C., Rodríguez, D.L., Bravo, L.E., & Silva, J.A. (2022). A review on the prediction of students' academic performance using ensemble methods.

[15]Dietterich, T.G. (2000). Ensemble Methods in Machine Learning. In: Multiple Classifier Systems. MCS 2000. Lecture Notes in Computer Science, vol 1857. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-45014-9_1

APPENDIX

1. App8.py

```
from flask import Flask, render_template, request, send_file
import pandas as pd
import joblib
from reportlab.pdfgen import canvas
from datetime import datetime
import os
import sqlite3

# Initialize the Flask app
app = Flask(__name__)

# Load the trained model
model = joblib.load("performance_model.pkl")

# Define the features expected in the input
FEATURES = ["Hours Studied", "Previous Scores_scaled", "Extracurricular Activities",
            "Sleep Hours", "Sample Question Papers Practiced"]

# Create a directory to store PDF reports if it doesn't exist
REPORTS_DIR = "reports"
os.makedirs(REPORTS_DIR, exist_ok=True)

# Initialize the SQLite database
def init_db():
    conn = sqlite3.connect("student_data.db")
    cursor = conn.cursor()
    cursor.execute("""
        CREATE TABLE IF NOT EXISTS student_entries (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            name TEXT,
            hours_studied REAL,
            previous_scores_scaled REAL,
            extracurricular_activities REAL,
            sleep_hours REAL,
            sample_question_papers_practiced REAL,
            predicted_class INTEGER,
            timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conn.commit()
    conn.close()

init_db()

@app.route("/")
def home():
    """Render the index page where users can input data."""
```

```

return render_template('index5.html')

@app.route("/predict", methods=["POST"])
def predict():
    try:
        # Initialize data dictionary
        data = {}

        # Check if form data is received
        if request.form:
            data = request.form.to_dict()
            # Convert inputs to float, handle conversion errors
            try:
                data_float = {key: float(value) for key, value in data.items() if key != "Name"}
            except ValueError:
                return "Invalid input, unable to convert to float", 400

        # Ensure all required features are present
        if not all(feature in data_float for feature in FEATURES):
            return f"Missing one or more required features: {FEATURES}", 400

        # Get student's name (if provided)
        student_name = request.form.get("Name", "Student")

        # Create a DataFrame for the input
        input_data = pd.DataFrame([data_float])

        # Make predictions
        prediction = model.predict(input_data)
        predicted_class = int(prediction[0])

        # Save data to the database
        save_to_db(student_name, data_float, predicted_class)

        # Generate detailed guidance based on the prediction
        guidance = generate_guidance(predicted_class)

        # Generate PDF report
        pdf_filename = generate_pdf(student_name, guidance, predicted_class)

        # Return the generated PDF file for download
        return send_file(pdf_filename, as_attachment=True)

    except Exception as e:
        return f"Error: {str(e)}", 500

def save_to_db(name, data, predicted_class):
    """Save user input and prediction to the SQLite database."""
    conn = sqlite3.connect("student_data.db")
    cursor = conn.cursor()
    cursor.execute("
        INSERT INTO student_entries (
            name, hours_studied, previous_scores_scaled, extracurricular_activities,
            sleep_hours, sample_question_papers_practiced, predicted_class
    ")

```

```

    ) VALUES (?, ?, ?, ?, ?, ?, ?)
", (
    name,
    data.get("Hours Studied"),
    data.get("Previous Scores_scaled"),
    data.get("Extracurricular Activities"),
    data.get("Sleep Hours"),
    data.get("Sample Question Papers Practiced"),
    predicted_class
))
conn.commit()
conn.close()

```

```

def generate_guidance(predicted_class):
    """Generate personalized guidance based on the predicted class."""
    guidance_map = {
        1: (
            "Focus on improving your study habits. Allocate consistent hours daily and review your notes regularly.\n"
            "Here are some actionable steps:\n"
            "- Create a structured daily schedule for study and breaks.\n"
            "- Join study groups to enhance collaborative learning.\n"
            "- Seek help from teachers or tutors for difficult topics.\n"
            "- Reduce distractions by setting up a dedicated study space."
        ),
        2: (
            "Good work! Keep practicing additional sample papers to strengthen your preparation.\n"
            "Additional tips for further improvement:\n"
            "- Review mistakes from previous tests to avoid repeating them.\n"
            "- Increase focus on time management during tests.\n"
            "- Explore advanced resources to challenge yourself.\n"
            "- Maintain a healthy balance between study and relaxation."
        ),
        3: (
            "Excellent performance! Consider helping peers to reinforce your understanding and improve leadership skills.\n"
            "Suggestions to excel further:\n"
            "- Take on challenging projects or competitions.\n"
            "- Mentor others to solidify your knowledge.\n"
            "- Stay updated with new developments in your field of interest.\n"
            "- Keep a journal to track your progress and set new goals."
        ),
    }

    return guidance_map.get(predicted_class, "Maintain a balanced approach to your studies and extracurricular activities.")

```

```

def generate_pdf(student_name, guidance, predicted_class):
    """Generate a detailed PDF report."""
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    pdf_filename = os.path.join(REPORTS_DIR, f'student_report_{timestamp}.pdf')

    c = canvas.Canvas(pdf_filename)

    # Add content to the PDF
    c.setFont("Helvetica-Bold", 16)
    c.drawString(100, 750, f'Student Performance Report for {student_name}')

    c.setFont("Helvetica", 12)
    c.drawString(100, 700, f'Predicted Performance Level: {predicted_class}')

    c.drawString(100, 650, "Personalized Guidance:")
    c.drawString(120, 630, guidance)

    c.drawString(100, 580, "Generated on: " + datetime.now().strftime("%Y-%m-%d %H:%M:%S"))

    c.save()

    return pdf_filename

if __name__ == "__main__":
    app.run(debug=True)

```

2.Index5.html

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Student Performance Prediction</title>
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/css/bootstrap.min.css">
</head>
<body>
<div class="container mt-5">
<h1 class="text-center">Student Performance Prediction</h1>
<form action="/predict" method="POST" class="mt-4">
<div class="mb-3">
<label for="name" class="form-label">Student Name</label>
<input type="text" id="name" name="Name" class="form-control"
placeholder="Enter your name" required>
</div>
<div class="mb-3">
<label for="hours-studied" class="form-label">Hours Studied</label>
<input type="number" step="0.1" id="hours-studied" name="Hours Studied"
class="form-control" placeholder="e.g., 5" required>

```

```

</div>
<div class="mb-3">
<label for="previous-scores" class="form-label">Previous Scores (scaled)</label>
<input type="number" step="0.01" id="previous-scores" name="Previous
Scores_scaled" class="form-control" placeholder="e.g., 0.85" required>
</div>
<div class="mb-3">
<label for="extracurricular" class="form-label">Extracurricular Activities
(hours/week)</label>
<input type="number" step="0.1" id="extracurricular" name="Extracurricular
Activities" class="form-control" placeholder="e.g., 2" required>
</div>
<div class="mb-3">
<label for="sleep-hours" class="form-label">Sleep Hours (per day)</label>
<input type="number" step="0.1" id="sleep-hours" name="Sleep Hours" class="form
control" placeholder="e.g., 7" required>
</div>
<div class="mb-3">
<label for="sample-questions" class="form-label">Sample Question Papers
Practiced</label>
<input type="number" step="1" id="sample-questions" name="Sample Question
Papers Practiced" class="form-control" placeholder="e.g., 3" required>
30</div>
<button type="submit" class="btn btn-primary">Predict Performance</button>
</form>
</div>
<script
src="https://cdnjs.cloudflare.com/ajax/libs/bootstrap/5.3.0/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

3. ALGORITHM(MODEL) FOR GENERATING PDF GUIDANCE REPORT

```

!pip install scikit-learn==1.3.2
import pandas as pd
import numpy as np
import seaborn as sns
import sklearn
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler,OneHotEncoder
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score
from sklearn.model_selection import learning_curve

```

```

!pip install xgboost
from xgboost import XGBRegressor

```

```

import matplotlib.pyplot as plt

```

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import joblib # For saving the model and scaler

# Define feature columns and target column
features = ["Hours Studied", "Previous Scores_scaled", "Extracurricular Activities",
            "Sleep Hours", "Sample Question Papers Practiced"]
target = "Performance_bin"

# Split the data into features (X) and target (y)
X = df[features]
y = df[target]

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Train a Random Forest Classifier
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)

# Evaluate the model
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:", accuracy)
print("\nClassification Report:")

print(classification_report(y_test, y_pred))

# Save the trained model and scaler
model_path = "performance_model.pkl"
#scaler_path = "scaler.pkl"

joblib.dump(model, model_path)
#joblib.dump(scaler, scaler_path)

print(f"Model saved to {model_path}")

```